

When an API URL returns JSON data, it is **JavaScript (JS), not HTML, that understands and processes the data during full-stack web development**. HTML is a markup language for structuring content, while JavaScript provides the interactivity to fetch, parse, and display that data dynamically. [1, 2, 3, 4, 5]

Here's how the process typically works in a full-stack application using a backend framework like Flask:

1. The Backend (Flask)

Your backend (e.g., in Python with Flask) handles the API request.

- You define an endpoint (a specific URL) that your application listens to.
- When that endpoint is requested, your Python code prepares data (often from a database) in a Python data structure like a dictionary or list.
- Flask's function (or automatic conversion in recent versions) serializes this Python data into a JSON formatted string, which is then sent back to the client as the API response. [6, 9, 10, 11, 12]

2. The Frontend (HTML and JavaScript)

The client-side (the user's web browser, running HTML, CSS, and JavaScript) receives the response.

- **HTML** defines the structure of your webpage (e.g., a where you want data to appear, or a for a list of items).
- **JavaScript** is the key component that communicates with the API. It uses modern features like the Fetch API to make an asynchronous request (AJAX call) to your Flask endpoint.
- When the JSON response is received, JavaScript:
 - Reads the raw response and uses the method to parse the JSON string into a usable JavaScript object.
 - Accesses individual data points using key-value pairs (dot notation).
 - Dynamically generates or updates the HTML content of the webpage to display the data to the user, typically by manipulating the Document Object Model (DOM). [1, 3, 18, 19, 20, 21, 22]

In summary, the backend provides the raw data in a standardized JSON format, and the frontend JavaScript translates that data into the visual HTML elements that the user sees in their browser. [23]

AI can make mistakes, so double-check responses

[1] <https://www.quora.com/What-is-JSON-and-how-does-it-relate-to-HTML-CSS>

[2] https://www.w3schools.com/js/js_json_html.asp

[3] <https://www.youtube.com/watch?v=37vxWr0WgQk>

[4] <https://hng.tech/learn/learn-javascript-programming-online-with-step-by-step-video-tutorials/w>

[orking-with-json-and-apis](#)

- [5] <https://www.linkedin.com/advice/1/how-can-you-dynamically-display-data-from-apis-bs4kc>
- [6] https://www.youtube.com/watch?v=VzBtoA_8qm4
- [7] <https://www.youtube.com/watch?v=uBiQEL43AQU>
- [8] https://www.reddit.com/r/flask/comments/o5wrt6/what_is_better_for_rendering_queried_data/
- [9] <https://www.youtube.com/watch?v=kvux1SiRIJQ>
- [10] <https://flask.palletsprojects.com/en/stable/quickstart/>
- [11] <https://codesignal.com/learn/courses/introduction-to-flask-basics/lessons/returning-json-for-structured-responses>
- [12] <https://apipheny.io/api-url-path/>
- [13] <https://web.dev/articles/client-side-rendering-of-html-and-interactivity>
- [14] <https://community.ibm.com/community/user/blogs/stephanie-wilkerson1/2020/12/17/write-web-applications-for-the-blueworks-live-rest>
- [15] <https://muthu.co/client-side-caching-of-api-responses/>
- [16] <https://aplاندeo.com/blog/single-page-applications-rest-better-dynamic-web-pages/>
- [17] <https://stackoverflow.com/questions/62329411/how-to-see-whether-a-website-is-client-side-rendered-or-server-side-rendered>
- [18] <https://rahulbaran.hashnode.dev/how-to-send-json-from-javascript-to-flask-using-fetch-api>
- [19] <https://www.youtube.com/watch?v=Oage6H4GX2o>
- [20] <https://reqbin.com/code/javascript/wc3qbk0b/javascript-fetch-json-example>
- [21] https://www.frontendmentor.io/articles/integrating-with-apis-a-beginners-guide-for-frontend-developers-u_xfEadBc-
- [22] <https://www.youtube.com/watch?v=uxf0--uiX0I>
- [23] <https://www.tencentcloud.com/techpedia/127972>